TEXT ANALYTICS

Data Gathering and Annotation

Francesco Spinnato

* francesco.spinnato@unipi.it
University of Pisa



Where Do Datasets Come From?

So far, every corpus was **given**: someone had already collected the texts and attached the labels.

In a real project, building the dataset is often the largest part of the work:

1. define the **information need**
2. **collect** the texts
3. **clean** and **inspect** them
4. **annotate** them, if the task is supervised
5. train a model and **evaluate** it

The model can only be as good as the data.

Data Gathering

From an information need to a corpus

Start From a Question

"Which aspects of their stay do hotel guests complain about most?"

Before writing any code:

- **which kind of texts** answer the question? Reviews, but from which platforms, which hotels, which period, which languages?
- **which unit** is a document? A review, a sentence, an aspect mention?
- **how many** do we need, and what will collecting them cost (time, storage, requests, money)?
- **are we allowed** to collect them, and to use them for this purpose?

The answers decide the source, the collection method, and later the annotation scheme.

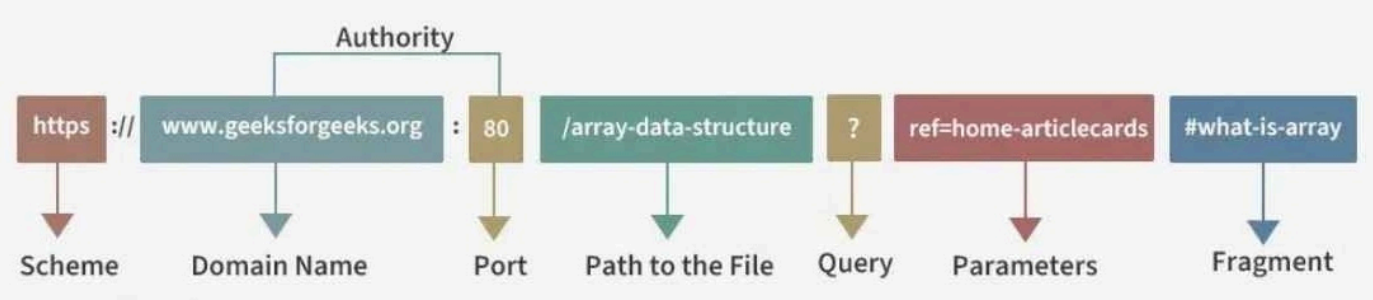
Where Text Comes From

Source	Examples	Labels included?
Existing datasets	Hugging Face Hub, Kaggle, shared tasks (SemEval, CLEF)	often
APIs	Wikipedia, arXiv, Stack Exchange, news providers	metadata only
Web feeds	RSS and Atom feeds of news sites and blogs	categories, tags
Scraping	any web page, when nothing better exists	whatever the page shows
Internal data	customer emails, support tickets, surveys	sometimes
Generated	text written by an LLM on request	the prompt

Look for an **existing dataset first**: it is cheaper, comparable with published results, and someone already solved the legal questions. Check its **license** and **where its labels come from**.

A Few Definitions

- **HTTP** is how programs ask a server for something:
  - a **request** (a URL, a method such as `GET`, headers such as the `User-Agent`) and
  - a **response** (a status code, then the content). Common codes: `200` OK, `403` forbidden, `404` not found, `429` too many requests.



- **HTML** is the language of web pages: text marked up with tags, written for a browser to **display** to people.
- An **API** (application programming interface) is a set of URLs a service offers to **programs**: they return data instead of pages.
- **JSON** is the usual format of that data: objects `{"key": value}`, lists `[...]`, strings and numbers.

APIs

When a service offers an API, prefer it to scraping:

- stable format, documented fields
- explicit terms of use
- **rate limits**: a maximum number of requests per time window
- often a **key**, and **pagination** for long results
- its own **usage rules**: who may call it, how often, for what

```python
import requests

url = "https://en.wikipedia.org/w/api.php"
headers = {"User-Agent": "TA-course (unipi.it)"}
params = {"action": "query", "format": "json",
          "titles": "Leaning Tower of Pisa",
          "prop": "extracts", "exintro": 1,
          "explaintext": 1}

pages = requests.get(url, params=params,
                     headers=headers).json()["query"]["pages"]
print(next(iter(pages.values()))["extract"])
```

```
The Leaning Tower of Pisa (Italian: torre
pendente di Pisa [...]), or simply the Tower
of Pisa, is the campanile, or freestanding
bell tower, of Pisa Cathedral...
```

APIs belong to the platform. In 2023, Twitter/X and Reddit restricted free access to theirs, and many research projects stopped overnight.

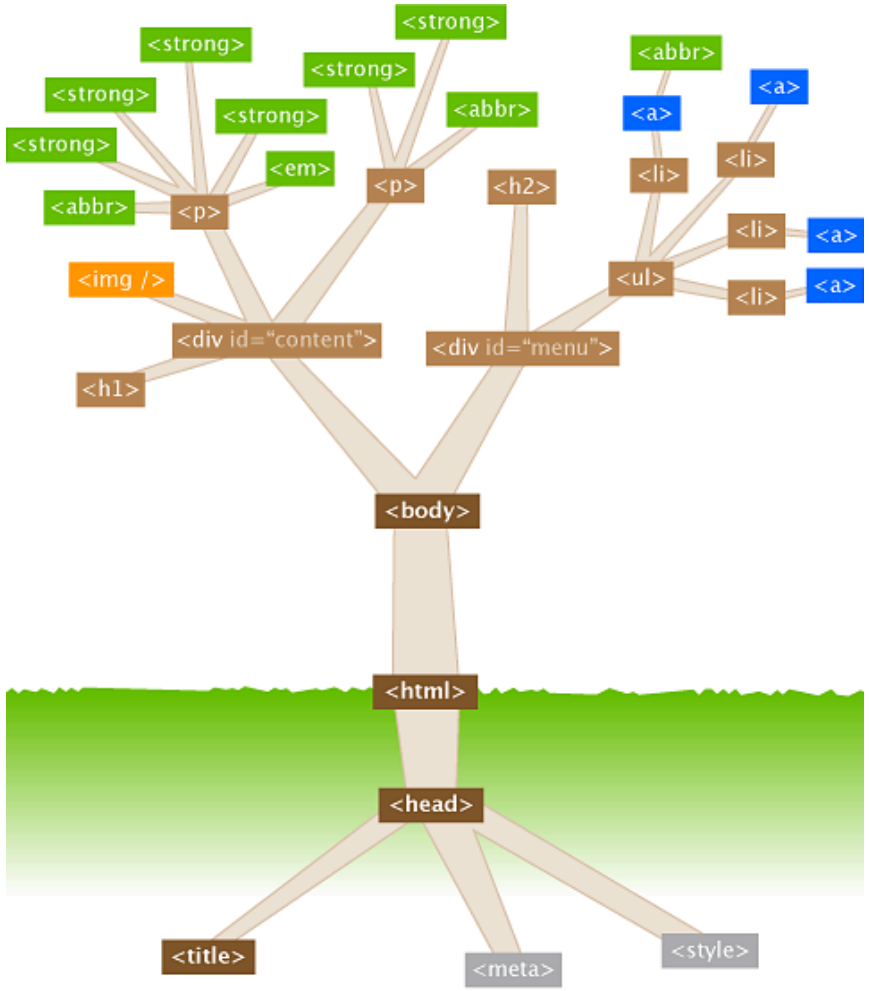
Scraping

When there is no API, we download the **HTML** pages and extract the text ourselves.

Scraping is mostly a matter of:

- finding the **URL scheme**: `/wiki/<title>`, `/page-2.html`, `?page=3`...
- collecting URLs from **index pages** (lists, categories, search results), then visiting each one
- **parsing** the HTML to keep only the parts we need

A page is a tree of **elements**. Each has a tag, attributes (`id`, `class`) and children.



Crawlers and robots.txt

- A **crawler** discovers pages automatically, by following links from page to page.
- A **scraper** extracts data from the pages it downloads. Most projects do both.

Sites say which paths crawlers may visit in a file at their root, `/robots.txt`:

```
# Apply these rules to all crawlers.
User-agent: *

# Allow this specific mobile-view API URL pattern, despite the broader /w/ rule below.
Allow: /w/api.php?action=mobileview&

# Ask crawlers not to crawl URLs under /w/ or /api/.
Disallow: /w/
Disallow: /api/

# Ask crawlers not to crawl Special pages under /wiki/.
Disallow: /wiki/Special:
```

Be a Polite Scraper

A scraper sends requests to a server that someone else pays for.

- respect **robots.txt**, and the site's terms of service
- **pause** between requests (one or two seconds), and never send them in parallel to the same site
- identify yourself with a descriptive **User-Agent**, with a way to contact you
- **cache** what you download, so that rerunning the code does not download it again
- stop when you have **enough**

Can I Use This Data?

Being able to download a text does not mean we may use it.

- **Terms of service**: many platforms forbid scraping, even of public pages.
- **Copyright and licenses**: Wikipedia text is CC BY-SA, so it can be reused with attribution; most news articles and books cannot be redistributed.
- **Personal data**: posts, reviews and comments are written by people. In the EU the GDPR applies to them even when they are public: collect only what you need, pseudonymize, store safely.
- **Text and data mining**: the EU Copyright Directive (2019/790) allows text and data mining for research, with conditions; outside research, rights holders can opt out.
- **Personal use**: a private copy for your own use is often allowed, but it ends when you **share**: a dataset on GitHub, given to classmates or used in a published report is no longer personal.

This is a general overview, not legal advice: the rules differ between countries and change over time.

Licenses

A **license** says what others may do with a work. **No license** means all rights reserved: you may not reuse it.

Most open data uses **Creative Commons** licenses, built from a few conditions:

- **BY**: credit the source
- **SA**: share what you derive under the same license
- **NC**: no commercial use

CC0 means no conditions at all. Wikipedia text is **CC BY-SA**.

A dataset's license covers only what its authors own: scraped reviews published as "open" are still the reviewers' texts.

Personal Data and the GDPR

In the EU, the **General Data Protection Regulation** (GDPR, 2016/679) protects **personal data**: any information about an identified or identifiable person. In text, that includes names, usernames, and details that together point to someone.

- **Public $\neq$ free**: the GDPR applies to posts and reviews that anyone can read.
- **Legal basis**: processing needs one, such as consent, a legitimate interest, or scientific research with safeguards.
- **Minimization and purpose**: collect only what the task needs, and use it only for the declared purpose.
- **Pseudonymized data is still personal data**: replacing names with IDs is a safeguard, not an exemption. Fully anonymizing text is hard: the content itself can identify the author.

Wikipedia biographies of living people are personal data: public figures are also protected by the GDPR.

Cleaning Web Text

Raw web text is never ready to use:

Problem	Example
Boilerplate	menus, cookie banners, "Share on Facebook", footers
Markup left in the text	citation markers `[3]`, image captions, tables flattened into text
Encoding errors	`donâ€™t` instead of `don't`: UTF-8 bytes decoded as Latin-1
Duplicates	the same article syndicated on several sites, reposts, quotes
Mixed languages	an English corpus with Spanish and German pages

Decide the cleaning **before** annotating: annotators should see the same text the model will see.

Inspect the Data

Before annotating or training, **read** a random sample, and count:

- **how many** documents, and how many are empty or very short
- the **length** distribution, in words
- **exact and near duplicates**
- the distribution of every **metadata** field: source, date, category, language

Questions to ask:

- does the data contain the information we need?
- is it **complete**, **clean**, **correct**?
- is it correctly linked to the entities we care about (the right hotel, the right product)?

Sampling and Bias

A corpus is a **sample**: a model learns what is in it.

- **Platform**: reviews from one site describe its users, not all customers.
- **Time**: a corpus collected in one month reflects that month's events and vocabulary.
- **Selection**: collecting tweets by keyword finds only the texts that use that keyword.
- **Volume**: a few very active authors can dominate a corpus.

These biases do not show up in a test score: the test set comes from the same sample. They show up in deployment.

Bias Beyond Sampling

Even a perfect sample carries the biases of the texts and of the labels:

- **Historical bias**: texts reflect the society that wrote them. Biographies describe a world where most painters and scientists were men, and a model learns the association between occupation and gender.
- **Coverage bias**: some groups are written about less. On the English Wikipedia, about one biography in five is about a woman.
- **Annotator bias**: labels reflect who annotates. Hate speech annotators have been shown to flag African American English as offensive more often, and models trained on their labels inherit it.
- **Labeler bias**: automatic labels inherit the blind spots of the rules or of the model that produced them.
- **Shortcuts**: features that correlate with the label in this corpus, but not in general: names, dates, the source of the text.

Document the Dataset

A dataset without documentation cannot be reused, or even trusted.
Datasheets for datasets propose a fixed list of questions:

- **motivation**: why was it created, and by whom?
- **composition**: what is a document, how many are there, what is missing?
- **collection**: which sources, which period, which filters?
- **labeling**: who annotated, with which guidelines, with which agreement?
- **uses and distribution**: license, intended uses, uses to avoid

For text, **data statements** add the language variety, and the demographics of speakers and annotators.

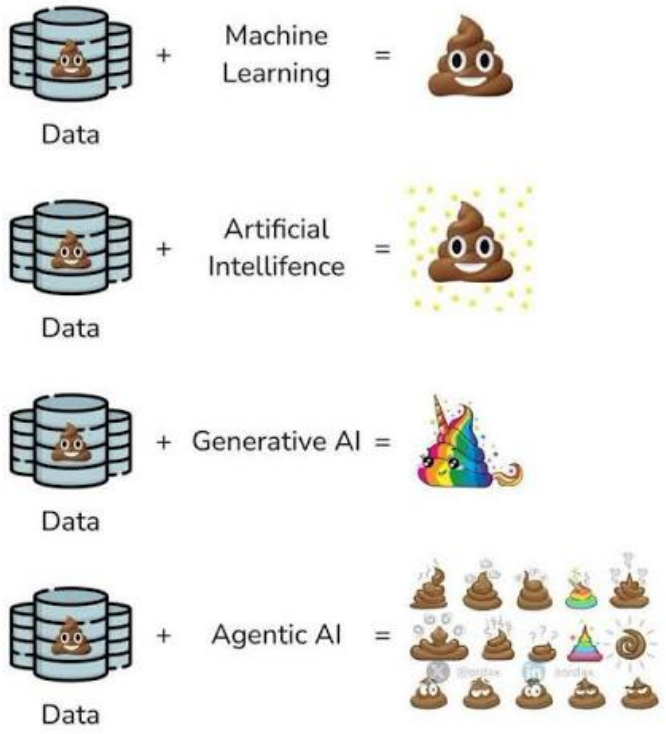
Data Annotation

Making the supervision explicit

Labeling Data

For supervised tasks, the labels are usually **not in the data**:

- the sentiment of a review is not marked explicitly
- spam tries to hide its nature
- the aspects discussed in a review are implicit in what it says
- entities are not marked in the text



Annotators make this information explicit, producing the training and test data of the model.

Garbage in, garbage out: the model can at best learn what the labels say. If the labels are noisy or inconsistent, so is the model, and so is its evaluation.

What Can Be Annotated

Unit	Annotation	Example
Document	one label	sentiment of a review, topic of an article
Document	several labels	the aspects a review mentions
Document	free text	a summary, a translation
Span	a label on a sequence of tokens	named entities: *Leaning Tower of Pisa* = LOCATION
Pair of spans	a relation	*Galileo* **born in** *Pisa*
Pair of documents	a preference or a similarity	which of two answers is more helpful?

Span and relation labels need tools that show the text and let annotators select parts of it. Document labels can be collected in a spreadsheet.

Annotation Guidelines

Two annotators reading the same review will not agree unless they share the **same definitions**. Guidelines contain:

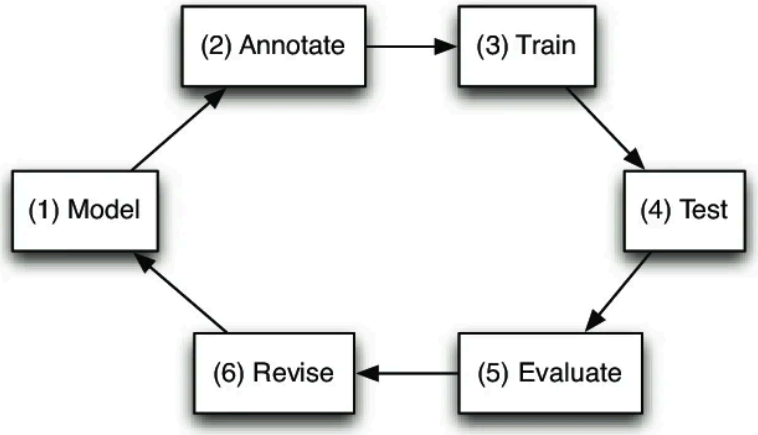
- the **label set**, with a definition of each label
- **decision rules** for the hard cases: sarcasm, mixed opinions, off-topic texts
- **examples** and counterexamples for every label
- what to do when unsure: an explicit **"unclear"** label is better than a forced guess

Guidelines are **iterated**: annotate a pilot sample, discuss the disagreements, revise the rules, and repeat until agreement stops improving.

The Annotation Cycle

Annotation for machine learning is a loop: the **MATTER** cycle:

1. **Model**: define the phenomenon and the label set
2. **Annotate**: write guidelines, annotate a sample
3. **Train**: fit a model on the annotations
4. **Test**: apply it to held-out annotated data
5. **Evaluate**: measure its performance, and the annotators' agreement
6. **Revise**: change the model of the phenomenon, the guidelines, or both, and start again



Errors of the model are often errors of the **label set**: two labels that annotators cannot tell apart cannot be learned either.

Use a development set for revisions; reserve the test set for final evaluation.

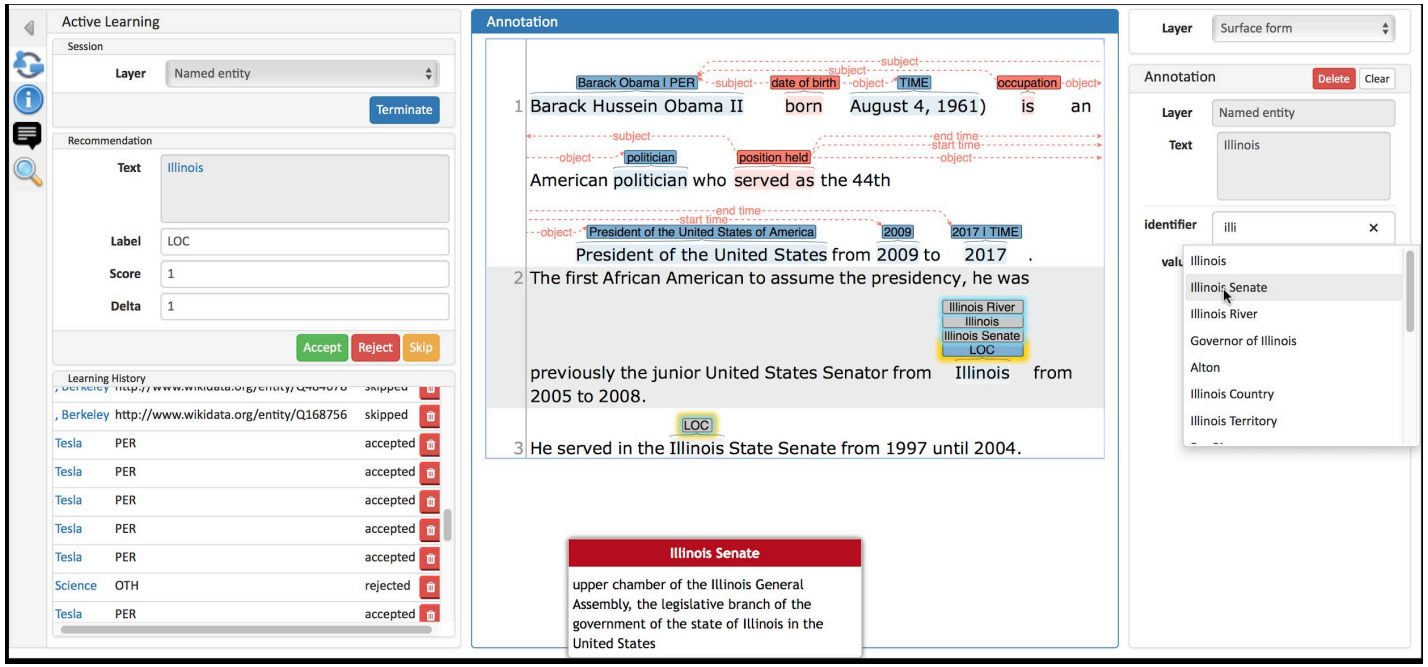
Who Annotates?

Annotators	Strengths	Weaknesses
Domain experts	correct on difficult texts (medical, legal)	expensive, few
Trained in-house annotators	consistent, can discuss the guidelines	slow to scale
Crowd workers (Prolific, MTurk)	fast, cheap, many labels per item	noisy: needs redundancy and quality checks
LLMs	fastest, cheapest	consistent errors, unknown biases, must be validated

Annotation Tools

What to look for:

- the annotation types you need (labels, spans, relations)
- several annotators per item, and assignment of items
- an **export** format you can read in Python



INCEpTION: entities, relations and links to Wikidata.

Annotation Tools: Some Options

Tool	What it annotates	Notes
Spreadsheet	document labels	no installation; enough for one label per document
doccano	labels, spans, relations, text-to-text	open source, web based, several annotators per project
Label Studio	labels, spans, relations; also images, audio, video	open source, web based, configurable interfaces
INCEpTION	spans, relations, links to knowledge bases	open source, suggestions from a model while annotating
Prodigy	labels, spans, relations	commercial, by the makers of spaCy, built around active learning

Keep track of **who** gave each label, with one file or one field per annotator.

These are just examples, anything that export formats Python can read (JSON, CSV) it's ok.

Inter-annotator Agreement

Whenever possible, more than one annotator labels the data:

- **together**, on an initial set, to agree on how to apply the guidelines
- **separately**, on a **shared** set, to measure their agreement
- on **distinct** sets, to cover more documents with the same effort

How much annotators agree tells us how well defined the task is. A model is trained to reproduce the labels: it cannot be more consistent than the people who produced them.

Agreement is an estimate of the best performance we can expect, although models do sometimes exceed it.

Raw Agreement Is Not Enough

Two annotators label 100 emails as spam or not:

	B: spam	**B: not spam**
A: spam	1	4
A: not spam	4	91

They agree on **92%** of the emails. Good?

- Each of them labels only 5% as spam.
- Two annotators who **never read** the emails, and label 5% of them as spam at random, agree about **90.5%** of the time, almost always on *not spam*.
- The 92% is almost entirely **chance agreement**: on the only class that matters, they agree on 1 email out of 9.

Cohen's Kappa

Cohen's kappa corrects the observed agreement for the agreement expected by chance:

$$\kappa = \frac{p_o - p_e}{1 - p_e} \qquad p_e = \sum_{c \in Y} p_A(c)\, p_B(c)$$

- p_o: the fraction of documents on which the two annotators agree
- $p_A(c)$, $p_B(c)$: the fraction of documents each annotator labels c
- p_e: the agreement of two annotators who label at random, each with their own label proportions

$\kappa = 1$: perfect agreement. $\kappa = 0$: no better than chance. $\kappa < 0$: worse than chance.

Kappa: Two Examples

Spam, the previous slide:

	B: spam	B: not
A: spam	1	4
A: not	4	91

$p_o = 0.92$

$p_e = 0.05 \cdot 0.05 + 0.95 \cdot 0.95 = 0.905$

$\kappa = \frac{0.92 - 0.905}{1 - 0.905} \approx \mathbf{0.16}$

Sentiment of 100 reviews:

	B: pos	B: neg
A: pos	40	10
A: neg	5	45

$p_o = 0.85$

$p_e = 0.50 \cdot 0.45 + 0.50 \cdot 0.55 = 0.50$

$\kappa = \frac{0.85 - 0.50}{1 - 0.50} = \mathbf{0.70}$

The sentiment annotators agree far beyond chance.

Reading Kappa

A commonly cited scale: **0.41 to 0.60** moderate, **0.61 to 0.80** substantial, above **0.80** almost perfect. The thresholds are conventions, not laws: subjective tasks (sarcasm, offensiveness) rarely reach 0.8.

When agreement is low, the cause can be:

- **unclear guidelines**: revise them, with examples of the disagreements
- **a label set that does not fit the data**: merge labels that annotators confuse
- **an ambiguous task**: some texts have no single right answer
- **careless annotators**: check them against a small set of known answers

With **more than two annotators**, or annotators who each label different subsets, use **Fleiss' kappa** or **Krippendorff's alpha**.

From Annotations to Labels

With several annotations per document, we need one label to train and evaluate on:

- **majority vote**: simple; needs an odd number of annotators, or a rule for ties
- **adjudication**: an expert resolves the disagreements; slower, cleaner
- **keep the disagreement**: the label distribution itself ("2 of 3 said positive") becomes a **soft label**, useful when the task is truly ambiguous

Always keep the **raw annotations**, with annotator IDs: they are needed to compute agreement, to find careless annotators, and to change the aggregation later.

Labels Without Annotators

Annotation is expensive, so labels are often obtained **automatically**:

- **distant supervision**: labels from metadata that comes with the text: star ratings for sentiment, hashtags or emoji for emotions, the section of a newspaper for the topic, the category of a Wikipedia page
- **rules and lexicons**: keyword lists, regular expressions, sentiment lexicons
- **models**: a pretrained classifier or an LLM labels the texts

These are **silver labels**: cheap and plentiful, but noisy, and biased toward their source.

Gold labels are produced by human annotators, ideally with measured agreement.

Many public datasets have silver labels, and do not say so: always check where the labels come from.

Gold and Silver

Silver labels are good for **training**, not for **testing**.

- Testing on silver labels measures **agreement with the labeler**, not correctness: a model trained on silver labels can at best imitate it.
- A score of 0.90 F1 against silver labels says nothing about how often the model is right.



What to do:

- annotate by hand a small **gold test set**, a few hundred documents
- evaluate **every** model on it, including the **labeler itself**:
  - it is one more annotator, and its agreement with the gold labels tells how far to trust the silver ones

Annotation Artifacts

The way data is collected and labeled can leave **shortcuts** in the text, which a model learns instead of the task:

- emotions labeled from texts like *"i feel **amazed**"*: the label word is in the text
- reviews that state their own rating: *"5 stars, would buy again"*
- in natural language inference, annotators asked to write a contradiction often add a negation, so *not* alone predicts the label

The model scores well on test data built the same way, and fails on real texts.

Read the most predictive features of a first model: if they are artifacts, the dataset, not the model, needs fixing.

A Checklist

Before training on a new dataset check:

1. the **information need**, the source and the unit of a document are written down
2. the collection is **legal**, **polite** and **reproducible** (code, dates, filters)
3. the texts are **cleaned** and **inspected**: duplicates, lengths, a random sample read
4. **guidelines** exist, and were revised on a pilot sample
5. **agreement** was measured on a shared sample, with kappa, not raw agreement
6. the **test set** has gold labels; silver labels are used only to train
7. the dataset is **documented**

Sources

Textbooks

- Pustejovsky & Stubbs (2012), *Natural Language Annotation for Machine Learning*, O'Reilly, ch. 1 (the MATTER cycle), 6 (agreement)